Feature	HyperApp	React	Vue.js	Angular
Size	Lightweight (~7KB)	Larger (~100KB)	Larger (~80KB)	Larger (~500KB)
Virtual DOM	Yes	Yes	Yes	Yes
Performance	Fast	Fast	Fast	Fast
Reactivity	Reactive state	Reactive state	Reactive state	Two-way binding
Learning curve	Easy	Moderate	Moderate	Steep
State management	Built-in	Redux (external)	Vuex (external)	Built-in
Component reusability	High	High	High	High
Community support	Growing	Vast	Vast	Vast
Extensibility	Modular	Plugin ecosystem	Plugin ecosystem	Module-based
Mobile support	Yes	Yes	Yes	Yes
Documentation	Well-documented	Comprehensive	Comprehensive	Comprehensive

HyperApp compared with other popular JS frameworks

Building a user interface with HyperApp

This lightweight JavaScript library for building user interfaces follows a declarative approach similar to React but with a much smaller footprint. Here's a step-by-step guide on how to build a user interface with HyperApp.

Step 1: Set up a new project

Start by creating a new HTML file and include the HyperApp library. You can either download the library and reference it locally or use a content delivery network (CDN) to include it in your project. Here's an example using the CDN:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>HyperApp UI</title>
<script src="https://unpkg.com/
hyperapp"></script>
</head>
<body>
<div id="app"></div>
<script src="app.js"></script>
</body>
</html>
```

Step 2: Define the initial state

In your JavaScript file (e.g., app.js), start by defining the initial state of your application. The state represents the data that will drive your UI. For example:

```
const state = {
};
count: 0
```

Step 3: Define actions

Actions in HyperApp are responsible for updating the state. They receive the current state as the first argument and can return a new state. Here's an example of an action that increments the count:

```
const actions = {
increment: () => (state) => ({
count: state.count + 1 })
};
```

Step 4: Define the view

The view describes the UI components and their structure. It's a function that returns a virtual DOM representation using HyperApp's JSX-like syntax.

```
const view  = (state, actions) => (
<div>
<h1>Count: {state.count}</h1>
<button onclick={actions.
increment)>Increment</button>
</div>
);
```

Step 5: Mount the application

Finally, mount the application by calling the 'app' function from HyperApp, passing the *state*, *actions*, and *view* as arguments. Here's an example:

```
const main = app(state, actions, view,
document.getElementById('app'));
```

That's it! A basic HyperApp application with a simple UI has been developed. Whenever the 'Increment' button is clicked, it will trigger the increment action, updating the *state* and re-rendering the *view*.

API request handling with HyperApp applications

Actions, effects, and asynchronous operations are frequently used in combination to handle API requests in HyperApp applications.